Part 1:

You are required to build a command line tool that converts a regular expression into its corresponding NFA **(Strictly)** using Thompson's construction algorithm. You must implement the algorithm yourself without using any libraries. The output should be **exactly as the algorithm output**, don't optimize by yourself. Your tool should run like this for example: convert "[A-Za-z]+[0-9]*"

Input format:

The input to the tool is simply a regular expression in text form. The expression may be invalid, so you need to validate it before transforming it first. Example of an invalid regular expression: [A-Za-z

The allowed regular expressions syntax is listed in the following table:

Input	Description	Example	Explanation
|	Or	a|b|c	Matches either a or b or c
(…)	Group	((ABC)|(abc))	Matches either the first group ABC or the second group abc
[…]	Match any character/range of characters within the bracket	[a-cA-C5]	Matches characters in the ranges a-c, A-C and 5
.	Any single character	a.c	Matches any alphanumeric string of length 3 that starts with a and ends with c
?	Zero or one repetition (Optional)	a?	Matches a or empty string
*	0 or more repetition	A*	Matches empty string or any number of As
+	1 or more repetition	A+	Matches any number of As

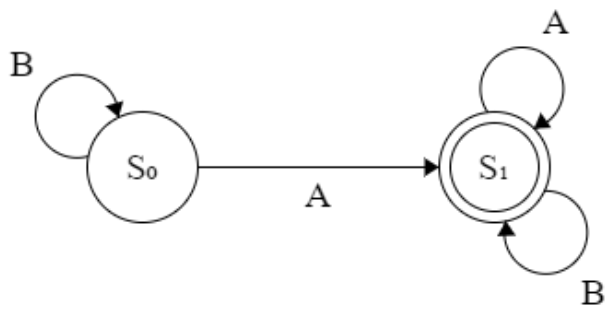
Note: input *literals* are alphanumeric. you **DO NOT** have to handle anything else.

Output format:

You are required to output a JSON file representing the FSM states and transition using the following format:

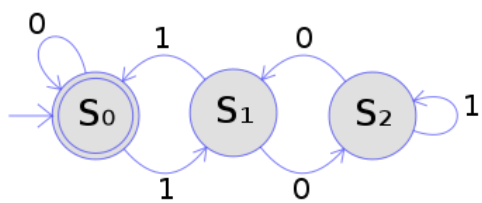
```
{
  "startingState": "S0",
  "S0": {
    "isTerminatingState": false,
    "A": "S1",
    "B": "S0"
  },
  "S1": {
    "isTerminatingState": true,
    "A": "S1",
    "B": "S1"
  }
}
```

Which corresponds to the following FSM



Example:

The following NFA



Would be corresponding to the following JSON:

```
{
  "startingState": "S0",
  "S0": {
    "isTerminatingState": true,
    "0": ["S0"],
    "1": ["S0", "S1"]
  },
  "S1": {
    "isTerminatingState": false,
    "0": ["S2"],
    "1": ["S0"]
  },
  "S2": {
    "isTerminatingState": false,
    "0": ["S1"],
    "1": ["S2"]
  }
}
```

You are also required to output an image containing the graph of the NFA, using any graphics library of your choice. You must distinguish between accepting and nonaccepting states. Neat and non-overlapping graphs are mandatory

Part 2:

You are required to implement a program that converts an NFA to a **minimized** DFA. You must implement the algorithm yourself without using any libraries.

Input format:

The input to the tool is a file containing the JSON representation of the NFA using the following format:

```
{
  "startingState": "S0",
  "S0": {
    "isTerminatingState": false,
    "A": "S1",
    "B": "S0"
  },
  "S1": {
    "isTerminatingState": true,
    "A": "S1",
    "B": "S1"
  }
}
```

Output format:

You are required to output a JSON file representing the **minimized** DFA states and transitions using the same format.

You are also required to output an image containing the graph of the **minimized** DFA, using any graphics library of your choice. You must distinguish between accepting and non-accepting states. Neat and non-overlapping graphs are mandatory.

Notes:

- You are required to follow the output format (json) as it is without any changes to it.
- The assignment will be in pairs.
- **Deadline**: exact deadline is to be posted on google classroom

Submission & Automated Grading Requirements

To facilitate automated grading, your submission must strictly adhere to the following project structure and execution guidelines. Failure to follow these formatting rules will result in the autograder failing your submission (& of course losing grades).

- Programming Language: The assignment must be implemented in Python.
- You are required to submit your work by providing a link to a GitHub repository containing your complete project. The repository must be private.
- Repo must have (requirements.txt) while the script will automatically pip install it
- Entry Point (main.py): Your repository must contain a file named main.py at the root level.
- Main Function Signature: Inside main.py, you must implement a function named process_regex with the exact signature below. The autograder will import this function to test your code.

```python
def process_regex(regex: str, out_name: str):
    # Your logic to:
    # 1. Validate the regex
    # 2. Convert regex to NFA (Thompson's construction)
    # 3. Convert NFA to Minimized DFA
    # 4. Generate and save the 4 required output files
    pass
```

- File Output Convention: When the autograder calls your function, it will provide a regular expression and a base name (e.g., process_regex("a|b", "test_case_1")). Your function must run the algorithms and generate exactly 4 outputs. These files must be saved in the same directory using the out_name parameter as a prefix:
    - <out_name>_nfa.json: The JSON for the NFA.
    - <out_name>_nfa.[ext]: The image for the NFA. Use (.png or .jpg) only.
    - <out_name>_dfa.json: The JSON for the minimized DF.
    - <out_name>_dfa.[ext]: The image for the minimized DFA. Use (.png or .jpg) only
- Here is the auto-grader link you have to make sure that your code will work on it https://colab.research.google.com/drive/1YKl4a4Y7YArONv4cmu_UfHatWNQJW8fy?usp=sharing (If any bug found, please report it early)